



BÁO CÁO THỰC TẬP

Khoa: Công nghệ thông tin

Giảng viên: PGS. TS. Nguyễn Ngọc Hóa

Đề tài: Dự án CoreRouter – 3S-COM (NFV-MANO Orchestrator)



Họ và tên: Hoàng Duy Hưng

Mã sinh viên: 22028293

Lớp: K67 – CS1

MAY 3, 2025

UNIVERSITY OF ENGINEERING AND TECHNOLOGY - VIETNAM NATIONAL UNIVERSITY HANOI

Mục lục

I. Giới thiệu chung	3
II. Giới thiệu công việc	4
III. Giới thiệu bài toán.....	5
IV. Yêu cầu bài toán	6
1. Yêu cầu chức năng:.....	6
2. Yêu cầu phi chức năng:.....	7
V. Tóm tắt lý thuyết, giải pháp, thuật toán	8
1. Kiến trúc NFV-MANO (ETSI NFV):.....	8
2. Nguyên lý SDN và tích hợp SDN Controller	9
3. Mô hình hóa dịch vụ mạng bằng TOSCA	9
4. Các nền tảng MANO nguồn mở	10
5. Công cụ triển khai tự động (Automation scripts)	10
VI. Cách giải quyết	11
1. Giai đoạn 1 – Thiết kế tài liệu (SRS).....	11
2. Giai đoạn 2 – Triển khai demo thử nghiệm (Mini-Lab)	12
VII. Mô tả phần mềm cài đặt	14
1. Môi trường và công cụ:.....	14
2. Đánh giá hoạt động	15
VIII. Kết quả đạt được và hướng phát triển.....	15
1. Kết quả đạt được:	16
2. Hạn chế và hướng phát triển	16
IX. Kỹ năng & kiến thức thu thập được.....	18
X. Hướng phát triển tiếp theo để hoàn thiện hệ thống	19
XI. Tài liệu tham khảo	21

LỜI CẢM ƠN

Em xin chân thành cảm ơn **Khoa Công nghệ Thông tin, Trường Đại học Công nghệ** đã tạo điều kiện cho Em được tham gia kỳ thực tập doanh nghiệp, qua đó có cơ hội tiếp cận, nghiên cứu và thực hành những kiến thức, kỹ năng chuyên môn trong môi trường thực tế.

Đặc biệt, Em xin gửi lời cảm ơn sâu sắc đến **Thầy Nguyễn Ngọc Hóa**, giảng viên hướng dẫn, người đã tận tình chỉ bảo, định hướng và động viên Em trong suốt quá trình thực tập. Sự hỗ trợ quý báu của Thầy/Cô đã giúp Em hiểu rõ hơn kiến thức lý thuyết, định hình phương pháp nghiên cứu, cũng như hoàn thành báo cáo thực tập này.

Em cũng xin gửi lời cảm ơn đến các anh/chị, bạn bè và đồng đội trong nhóm thực tập đã luôn đồng hành, trao đổi, chia sẻ kinh nghiệm và hỗ trợ Em trong suốt quá trình triển khai dự án.

Hà Nội, ngày 3 tháng 10 năm 2025

Sinh viên thực hiện

Hưng

Hoàng Duy Hưng

I. Giới thiệu chung

Dự án **CoreRouter** hướng đến xây dựng một nền tảng điều hành trung tâm cho mạng lõi thế hệ mới, cho phép điều phối linh hoạt các dịch vụ mạng dựa trên công nghệ **ảo hóa chức năng mạng (NFV)** và **mạng điều khiển bằng phần mềm (SDN)**. Trong đó, hệ thống **3S-COM** (*Smart, Secure and SDN&NFV-powered Controller, Orchestrator & Manager*) được đề xuất làm giải pháp quản lý và điều phối hợp nhất các chức năng mạng ảo (VNF) và dịch vụ mạng trong mạng lõi ảo hóa. 3S-COM đóng vai trò “bộ não” trung tâm của CoreRouter, phụ trách triển khai, cấu hình và quản lý vòng đời cho các VNF, đồng thời tích hợp điều khiển SDN để tối ưu hóa tài nguyên mạng. Hệ thống ban đầu được triển khai thử nghiệm trong môi trường lab, sau đó dự kiến mở rộng ra hạ tầng thực tế, đảm bảo các yêu cầu cao về bảo mật, độ sẵn sàng (HA) và hỗ trợ đa người dùng/đa **tenant**. Báo cáo này trình bày chi tiết quá trình thực tập tham gia dự án CoreRouter, tập trung vào các nhiệm vụ chính mà đã trực tiếp thực hiện, bao gồm: nghiên cứu lý thuyết nền tảng, thiết kế đặc tả yêu cầu hệ thống 3S-COM, và triển khai thử nghiệm một **Demo** nhỏ về Orchestrator NFV.

II. Giới thiệu công việc

Trong khuôn khổ dự án CoreRouter, tham gia với vai trò nghiên cứu và phát triển một phần hệ thống 3S-COM. Cụ thể, các công việc chính được giao gồm:

- **Nghiên cứu lý thuyết & viết tài liệu nền tảng:** Tìm hiểu các kiến thức cốt lõi liên quan đến NFV, SDN, mô hình hóa dịch vụ mạng và bảo mật mạng bằng AI... từ đó biên soạn tài liệu “**Kiến trúc nền tảng cho hệ thống 3S-COM**” làm cơ sở xây dựng hệ thống. Tài liệu này tóm tắt các chuẩn và công nghệ quan trọng như kiến trúc **NFV-MANO** theo ETSI, nguyên lý SDN và tích hợp SDN Controller, chuẩn **TOSCA** để mô hình hóa VNF/ dịch vụ, cũng như khả năng ứng dụng AI/ML trong phát hiện xâm nhập (IDS/IPS).
- **Xây dựng đặc tả yêu cầu hệ thống (SRS):** Thiết kế tài liệu “**Đặc tả yêu cầu hệ thống 3S-COM**” theo mẫu SRS chuẩn, định nghĩa rõ kiến trúc tổng quan, các yêu cầu chức năng và phi chức năng cho hệ thống. SRS này tuân thủ mô hình **ETSI NFV** và định hướng kiến trúc hệ thống theo các thành phần NFV-MANO, làm nền tảng cho giai đoạn thiết kế chi tiết và triển khai sau này.
- **Triển khai Demo 1 – Orchestrator NFV thu gọn:** Xây dựng một mô hình thí điểm (**mini-lab**) để minh họa nguyên lý hoạt động của hệ thống 3S-COM trong quản lý VNF. Demo sử dụng Kubernetes làm hạ tầng NFV, kết hợp một công cụ Orchestrator nhẹ (như **Open Source MANO (OSM)** hoặc script tự phát triển bằng **Ansible**) nhằm triển khai thử một VNF mẫu (ví dụ: một container firewall đơn giản hoặc router ảo **FRRouting**) và quản lý vòng đời của nó (khởi tạo, mở rộng, hủy) trên môi trường ảo hóa. Đây là bước thực nghiệm giúp kiểm chứng các ý tưởng cốt lõi trước khi tiến tới giải pháp đầy đủ.

Báo cáo này sẽ lần lượt trình bày bối cảnh bài toán, yêu cầu đặt ra, cơ sở lý thuyết liên quan, giải pháp mà thực hiện, mô tả chi tiết về demo triển khai, kết quả đạt được, những kỹ năng tích lũy và định hướng phát triển tiếp theo cho hệ thống.

III. Giới thiệu bài toán

Bài toán đặt ra xoay quanh nhu cầu **quản lý và điều phối tập trung các chức năng mạng ảo hóa** trong mạng lõi viễn thông hiện đại. Trước đây, các thiết bị mạng chuyên dụng (router, firewall, v.v.) được triển khai trên phần cứng cố định, dẫn đến chi phí cao và thiếu linh hoạt. Với sự ra đời của **NFV (Network Functions Virtualization)**, các chức năng mạng này có thể được triển khai dưới dạng phần mềm trên hạ tầng ảo (VM, container), cho phép tạo lập dịch vụ mạng nhanh hơn (từ vài tháng rút xuống vài giờ) và tối ưu sử dụng tài nguyên. Tuy nhiên, việc ảo hóa đặt ra thách thức mới: làm thế nào để **phối hợp quản lý hàng loạt VNF và tài nguyên ảo** một cách nhất quán, tự động, đặc biệt khi chúng thuộc nhiều nhà cung cấp và nền tảng khác nhau.

Trong bối cảnh đó, **ETSI** đã đề xuất kiến trúc **NFV-MANO (Management and Orchestration)** – bộ khung chuẩn cho phép quản lý dịch vụ mạng ảo hóa và hạ tầng NFV một cách thống nhất. Kiến trúc NFV-MANO gồm ba khối chức năng chính: **NFV Orchestrator (NFVO)**, **VNF Manager (VNFM)** và **Virtualized Infrastructure Manager (VIM)**, cùng các giao diện chuẩn giữa chúng. NFVO chịu trách nhiệm điều phối *Network Service* (tập hợp các VNF liên kết để cung cấp dịch vụ end-to-end) và quản lý phân bổ tài nguyên hạ tầng cho dịch vụ đó. VNFM quản lý trực tiếp vòng đời của từng VNF (khởi tạo, cấu hình, mở rộng, kết thúc). Còn VIM quản lý lớp hạ tầng ảo hóa (NFVI), bao gồm tài nguyên tính toán, lưu trữ, mạng – ví dụ cụm máy chủ Kubernetes hoặc cloud OpenStack – để cung cấp máy ảo/container cho các VNF. Ba thành phần này phối hợp chặt chẽ giúp triển khai đúng yêu cầu và quản lý toàn bộ vòng đời dịch vụ mạng, từ khâu định nghĩa, triển khai, cấu hình, mở rộng cho đến khi kết thúc dịch vụ.

Vấn đề đặt ra cho dự án 3S-COM là xây dựng được một hệ thống MANO hoàn chỉnh theo chuẩn ETSI NFV nói trên, làm nền tảng cho mạng lõi ảo hóa của CoreRouter. Cụ thể, hệ thống cần cho phép người quản trị **thiết kế và định nghĩa dịch vụ mạng** (gồm các VNF thành phần và kết nối giữa chúng), sau đó **tự động triển khai dịch vụ đó lên hạ tầng ảo**, đồng thời giám sát và điều chỉnh linh hoạt khi có thay đổi (tăng/giảm tài nguyên, thay thế VNF lỗi, mở rộng năng lực...). Tất cả phải diễn ra một cách **tập trung, thống nhất qua một “cổng” duy nhất**, giảm thiểu can thiệp thủ công và tránh sai sót. Bên cạnh đó, do mạng lõi yêu cầu tính ổn định và bảo mật cao, hệ thống cũng phải có khả năng **tích hợp cơ chế SDN** để điều khiển luồng mạng (ví dụ, chèn chính sách chặn lọc khi phát hiện tấn công), cũng như áp dụng **AI/ML** để phát hiện sớm các bất thường và tự động ứng phó (ví dụ tự động scale-out khi VNF quá tải hoặc cô lập phân đoạn mạng khi bị tấn công). Bài toán đặt ra đa chiều như vậy đòi hỏi 3S-COM phải được thiết kế bài bản, tuân thủ chuẩn, đồng thời đủ mở để tích hợp nhiều công nghệ (NFV, SDN, AI) vào một nền tảng thống nhất.

Tóm lại, mục tiêu cốt lõi của bài toán là: **xây dựng một hệ thống Orchestrator trung tâm cho mạng lõi ảo hóa**, cho phép quản lý mọi mặt của dịch vụ mạng (từ thiết kế, triển khai đến vận hành, tối ưu) một cách tự động, linh hoạt và thông minh. Phần tiếp theo sẽ liệt kê các yêu cầu cụ thể mà hệ thống 3S-COM cần đáp ứng để giải quyết bài toán này.

IV. Yêu cầu bài toán

Căn cứ trên bài toán và định hướng kiến trúc NFV-MANO, hệ thống 3S-COM được đề ra các yêu cầu chính như sau:

1. Yêu cầu chức năng:

- **Quản lý vòng đời VNF** – Hệ thống phải cho phép quản lý đầy đủ vòng đời của các **chức năng mạng ảo** trong mạng lõi. Cụ thể bao gồm khả năng khởi tạo VNF mới từ mẫu (onboard & instantiate), cấu hình khởi tạo cho VNF, **mở rộng hoặc thu hẹp** quy mô VNF khi cần thiết (hỗ trợ cả **scale-out/in** – tăng/giảm số instance và **scale-up/down** – tăng/giảm tài nguyên cho instance, tùy khả năng của VNF), cũng như tạm dừng, tiếp tục hoặc **kết thúc (terminate)** VNF khi không còn cần thiết. Việc mở rộng/v.s. thu hẹp có thể do người quản trị chủ động thực hiện hoặc do hệ thống tự động kích hoạt dựa trên chính sách (ví dụ, tự động scale-out khi tải vượt ngưỡng).
- **Điều phối dịch vụ mạng (Network Service)** – 3S-COM phải quản lý được các **dịch vụ mạng ảo hợp thành từ nhiều VNF**. Hệ thống cần có khả năng định nghĩa **mô hình dịch vụ** (bao gồm các VNF thành phần và liên kết giữa chúng), triển khai đồng bộ các VNF đó trên hạ tầng và thiết lập kết nối mạng phù hợp giữa chúng. Ví dụ, một dịch vụ VPN ảo có thể bao gồm 2 VNF (vRouter và vFirewall) nối tiếp; Orchestrator cần đảm bảo tạo cả hai VNF và nối chúng qua một mạng ảo nội bộ theo đúng mô hình. Khi có yêu cầu thay đổi dịch vụ (thêm VNF mới, thay đổi chuỗi), hệ thống cũng phải hỗ trợ cập nhật linh hoạt.
- **Quản lý tài nguyên hạ tầng (NFVI)** – Hệ thống phải theo dõi và quản lý hiệu quả **hạ tầng ảo hóa** phía dưới, bao gồm cụm máy chủ vật lý, các máy ảo/container đang chạy và tài nguyên mạng ảo. 3S-COM cần duy trì **inventory** đầy đủ về tài nguyên: danh sách các **node** (chủ vật lý/hosts), dung lượng tính toán, lưu trữ, mạng khả dụng, cũng như trạng thái hiện thời (số lượng VM/container đang chạy, tài nguyên đã sử dụng, cảnh báo nếu node nào down, v.v.). Trước mỗi yêu cầu triển khai VNF mới, hệ thống phải kiểm tra được tài nguyên còn đáp ứng đủ không. Ngoài ra, hệ thống cần quản lý các **mạng ảo** mà VNF sử dụng (network overlay, VLAN/VxLAN, địa chỉ IP cấp phát cho VNF) thông qua VIM. Tất cả tương tác với hạ tầng này về lý thuyết sẽ do VIM đảm nhiệm, nhưng 3S-COM phải cung cấp giao diện để quản trị viên xem và kiểm soát cần thiết (ví dụ hiển thị danh sách node Kubernetes, pods của các VNF, mạng dịch vụ...).
- **Tích hợp điều khiển SDN** – Một yêu cầu quan trọng là hệ thống có khả năng **quản lý SDN Controller** phục vụ các thiết bị mạng vật lý trong core mạng, đặc biệt là các bộ định tuyến lõi hiệu năng cao dùng công nghệ lập trình như ASIC **P4**. Cụ thể, 3S-COM phải cho phép cấu hình thông tin kết nối đến SDN Controller (thêm/sửa/xóa controller), giám sát trạng

thái kết nối giữa controller và các thiết bị (switch/router), cũng như gửi các lệnh điều khiển **flow** hoặc **policy** xuống controller khi cần thay đổi luồng mạng. Yêu cầu này nhằm đảm bảo 3S-COM có thể điều phối đồng thời cả **mạng ảo (các VNF)** lẫn **mạng vật lý** bên dưới, tạo thành một hệ thống hợp nhất. (Lưu ý: phạm vi thực tập chủ yếu tập trung vào phần NFV, do đó tích hợp SDN mới chỉ dừng ở mức thiết kế tài liệu, chưa triển khai trong demo).

- **Giám sát và quản lý cấu hình** – Hệ thống cần có chức năng **giám sát hiệu năng** của các VNF và hạ tầng, thu thập các chỉ số quan trọng (CPU, RAM, băng thông, logs sự kiện...) nhằm phát hiện sớm sự cố hoặc điểm quá tải. Đồng thời, 3S-COM phải hỗ trợ **quản lý cấu hình VNF từ xa**, cho phép người quản trị điều chỉnh tham số cấu hình hoặc cập nhật phần mềm trên VNF trực tiếp từ giao diện trung tâm mà không cần truy cập thủ công từng máy ảo/container. Chẳng hạn, quản trị viên có thể thay đổi rule của vFirewall hoặc cập nhật phiên bản cho vRouter thông qua 3S-COM, hệ thống sẽ tương tác với VNFM/VNF để áp dụng thay đổi. Ngoài ra, cần có cơ chế sao lưu cấu hình và khôi phục khi cần thiết (backup/restore) để nhanh chóng đưa VNF trở lại trạng thái ổn định sau sự cố.
- **Năng lực phân tích bảo mật (AI/ML)** – Để tăng tính chủ động trong bảo mật, 3S-COM được yêu cầu tích hợp mô-đun phân tích dữ liệu dựa trên **AI/ML** nhằm phát hiện **xâm nhập** hoặc bất thường trong luồng mạng. Hệ thống sẽ thu thập log từ các VNF bảo mật (như firewall, IDS) và thông tin lưu lượng từ hạ tầng, gửi cho mô hình AI phân tích. Nếu phát hiện dấu hiệu tấn công (ví dụ DDoS, xâm nhập trái phép) hoặc sự cố bất thường, kết quả sẽ được trả về 3S-COM để kích hoạt cảnh báo và thậm chí **tự động thực thi hành động ứng phó**. Ví dụ, khi phát hiện vFirewall bị quá tải do tấn công, hệ thống có thể tự động **scale-out** thêm một instance firewall mới và chia tải, hoặc khi nhận diện luồng độc hại, hệ thống tự động tạo luật chặn trên SDN controller. Yêu cầu này hướng tới xây dựng khả năng **tự vận hành (closed-loop automation)** cho mạng lõi: tự động nhận biết và phản ứng nhanh trước các tình huống, giảm thiểu phụ thuộc con người.

2. Yêu cầu phi chức năng:

- **Tuân thủ chuẩn mở** – Hệ thống phải tuân thủ các tiêu chuẩn mở của ETSI NFV và các API giao tiếp chuẩn (NFV-SOL005, SOL003, v.v.) để đảm bảo khả năng tương thích và mở rộng với nhiều loại VNF và hạ tầng khác nhau. Việc tuân thủ chuẩn giúp 3S-COM dễ dàng tích hợp module bên thứ ba (VD: dùng VNFM ngoài, OSS/BSS ngoài) và tránh bị khóa chặt vào một nhà cung cấp.
- **Hiệu năng và độ ổn định** – 3S-COM yêu cầu xử lý được nhiều yêu cầu điều phối song song trong thời gian thực, do đó hệ thống phải được thiết kế hiệu năng cao và có kiến trúc **micro-service** linh hoạt (như ONAP) để dễ dàng mở rộng theo tải. Đồng thời, độ ổn định và sẵn sàng cao (**HA**) là bắt buộc: hệ thống cần có khả năng chịu lỗi (failover) và không có điểm đơn lỗi, đảm bảo dịch vụ mạng không bị gián đoạn kể cả khi một thành phần của 3S-COM gặp sự cố.
- **Bảo mật** – Do là hệ thống quản lý trung tâm, 3S-COM phải đảm bảo an ninh chặt chẽ: giới hạn truy cập theo vai trò (RBAC), xác thực và mã hóa khi tương tác API, bảo vệ dữ liệu cấu hình nhạy cảm, và tích hợp giám sát an ninh (security logs, cảnh báo bất thường). Ngoài

ra, các VNF được điều phối cũng cần tuân theo chính sách bảo mật (ví dụ, chỉ cho phép image VNF đã được kiểm duyệt, cô lập các VNF thuộc tenant khác nhau...).

- **Trải nghiệm người dùng** – Hệ thống nên cung cấp giao diện quản lý trực quan (web portal) để người dùng dễ thao tác triển khai, theo dõi trạng thái mạng. Dù giai đoạn đầu có thể chỉ dùng công cụ dòng lệnh hoặc giao diện mặc định của nền tảng (như GUI của OSM), về lâu dài cần xây dựng **dashboard** tùy biến cho phép hiển thị tổng quan mạng lõi, bản đồ các VNF/ dịch vụ đang chạy, và thao tác quản trị nhanh chóng.

Các yêu cầu trên đây được tổng hợp dựa trên định hướng dự án và tham khảo từ các nền tảng MANO hiện có. Chúng đã được chi tiết hóa trong tài liệu SRS để làm cơ sở cho thiết kế hệ thống 3S-COM. Trong quá trình thực tập, đã tập trung vào các yêu cầu liên quan đến **quản lý VNF** và **NFVI**, đồng thời đảm bảo kiến trúc đề xuất tuân thủ NFV-MANO. Phần tiếp theo trình bày tóm tắt các kiến thức lý thuyết và giải pháp kỹ thuật mà đã nghiên cứu, làm nền tảng cho việc thực hiện hệ thống.

V. Tóm tắt lý thuyết, giải pháp, thuật toán

Để đáp ứng các yêu cầu nêu trên, việc thiết kế 3S-COM dựa trên tập hợp các công nghệ và giải pháp tiên tiến trong lĩnh vực mạng viễn thông. đã nghiên cứu các khía cạnh lý thuyết sau, từ đó đề xuất giải pháp phù hợp cho hệ thống:

1. Kiến trúc NFV-MANO (ETSI NFV):

Như đã giới thiệu, NFV-MANO cung cấp khung quản lý và điều phối các chức năng mạng ảo hóa một cách chuẩn hóa. ETSI định nghĩa 3 khối NFVO, VNFM, VIM với các chức năng và giao diện cụ thể. Trong hệ thống 3S-COM, nhóm thiết kế đã quyết định **tuân theo sát kiến trúc ETSI NFV** này nhằm đảm bảo tính tương thích và kế thừa được các nỗ lực phát triển có sẵn. Cụ thể, vai trò **NFVO/VNFM** sẽ được hiện thực hóa bởi một nền tảng phần mềm có sẵn, còn **VIM** sẽ tận dụng nền tảng ảo hóa phổ biến:

- **Lựa chọn NFVO/VNFM:** Sau khi khảo sát, dự án chọn sử dụng **ONAP (Open Network Automation Platform)** – một nền tảng nguồn mở do Linux Foundation phát triển – để đảm nhận vai trò NFVO và VNFM hợp nhất. ONAP cung cấp đầy đủ các module cần thiết cho điều phối dịch vụ, quản lý VNF, quản lý cấu hình, chính sách, phân tích dữ liệu... theo kiến trúc microservice chạy trên Kubernetes. Đây là giải pháp mạnh mẽ đã được ngành công nghiệp viễn thông chấp nhận, giúp 3S-COM thừa hưởng nhiều tính năng sẵn có (thay vì phải tự phát triển tất cả từ đầu). ONAP cũng hỗ trợ tốt việc tích hợp với chuẩn ETSI – ví dụ các phiên bản ONAP gần đây cho phép **onboard VNF** theo chuẩn **ETSI SOL001** (định dạng TOSCA VNFD) và cung cấp các API northbound tuân thủ **SOL005** để kết nối OSS/BSS.
- **Lựa chọn VIM:** Đối với lớp hạ tầng, dự án quyết định sử dụng **Kubernetes** làm VIM để quản lý các tài nguyên container và node vật lý/ảo. Kubernetes ngày càng được coi là một lựa chọn khả thi cho NFV nhờ hỗ trợ triển khai **CNF (Containerized Network Function)**

– các VNF dạng container – với ưu điểm nhẹ và nhanh hơn so với VM truyền thống. ONAP có sẵn module **Multi-VIM/Cloud (Multi-Cloud)** cho phép kết nối đến nhiều loại VIM, trong đó tích hợp Kubernetes như một VIM đặc biệt để quản lý container pods như là VNF. Cách tiếp cận ONAP + Kubernetes giúp hệ thống tận dụng được tính linh hoạt của cloud-native, triển khai VNF đồng nhất trên mọi hạ tầng (on-premise, đám mây) và tránh phụ thuộc vào một nhà cung cấp cụ thể.

Như vậy, về tổng thể kiến trúc logic, 3S-COM gồm các thành phần tương ứng NFVO/VNFM (chạy trên ONAP) tương tác với VIM (Kubernetes cluster) để quản lý VNF. Bên trên NFVO có thể tích hợp với OSS/BSS nếu cần (giao diện Os-Ma), bên dưới VIM tương tác hạ tầng vật lý. Hệ thống cũng có các cơ sở dữ liệu (danh mục VNF/NS, trạng thái tài nguyên...) và các thành phần phụ trợ (như module AI phân tích dữ liệu, SDN Controller) kết nối vào. Kiến trúc vật lý giai đoạn đầu của 3S-COM dự kiến triển khai tất cả thành phần trên một cụm máy chủ trong phòng lab để kiểm chứng, sau đó có thể tách ra nhiều node cho môi trường production.

2. Nguyên lý SDN và tích hợp SDN Controller

Song song với NFV, công nghệ **SDN (Software-Defined Networking)** được sử dụng để linh hoạt điều khiển hạ tầng mạng. SDN tách biệt mặt phẳng điều khiển (control plane) khỏi mặt phẳng dữ liệu (data plane) của thiết bị mạng, giúp việc quản lý tập trung thông qua **SDN Controller** trở nên khả thi. Trong phạm vi 3S-COM, SDN đóng vai trò kết nối thế giới ảo và vật lý: 3S-COM sẽ điều khiển các **thiết bị mạng vật lý khả lập trình (như router lõi dùng P4)** thông qua một SDN Controller tích hợp (dự kiến sử dụng ONOS hoặc OpenDaylight). đã nghiên cứu các giao thức southbound phổ biến (OpenFlow, P4Runtime) để hiểu cách controller giao tiếp với thiết bị P4, cũng như khảo sát các nền tảng SDN Controller mã nguồn mở. Kết quả định hướng rằng hệ thống sẽ tích hợp một **SDN Controller** như thành phần con: 3S-COM có thể gửi lệnh (thông qua API northbound của controller) để thiết lập **flow rule** hoặc **QoS policy** trên switch/router vật lý khi cần. Ví dụ, khi muốn cô lập một địa chỉ tấn công, 3S-COM sẽ ra lệnh cho SDN Controller chặn lưu lượng từ địa chỉ đó trên toàn mạng. Việc tích hợp SDN như vậy đã được đưa vào yêu cầu chức năng (phần quản lý SDN Controller) và sẽ là một phần của kiến trúc 3S-COM đầy đủ, dù trong demo hiện tại chưa triển khai.

3. Mô hình hóa dịch vụ mạng bằng TOSCA

Để Orchestrator có thể tự động triển khai và quản lý VNF/NS của nhiều nhà cung cấp khác nhau, cần một cách mô tả chuẩn hóa các **gói chức năng mạng**. ETSI NFV giải quyết vấn đề này bằng cách sử dụng chuẩn **TOSCA (Topology and Orchestration Specification for Cloud Applications)**. Cụ thể, ETSI đưa ra chuẩn **NFV-SOL001** – đặc tả mô hình thông tin cho **VNF Descriptor (VNFD)** và **Network Service Descriptor (NSD)** dưới dạng **TOSCA YAML**. Mỗi VNF/NS sẽ có một tập tin YAML mô tả chi tiết yêu cầu tài nguyên, các **VDU (Virtual Deployment Unit)**, kết nối mạng (**virtual links**), **interface** cho cấu hình, *lifecycle events* và *management scripts*, v.v. Bộ descriptor này được đóng gói thành gói CSAR (Cloud Service Archive) cùng với các script khởi tạo, file ảnh VM/container... để nhập vào Orchestrator. Trong quá trình nghiên cứu, đã tìm hiểu cấu trúc một **VNFD/NSD TOSCA** mẫu, các node types do ETSI định nghĩa sẵn (compute, network, VL, CP, etc.), cũng như cách ONAP sử dụng

chúng trong việc **onboard** VNF mới. Xu hướng hiện nay cũng mở rộng sang mô hình **CNF**: khi VNF chạy dạng container, descriptor TOSCA có thể chỉ định helm chart hoặc YAML Kubernetes để triển khai. Ví dụ, một VNFD có thể chứa **Helm chart** của ứng dụng và Orchestrator (ONAP, OSM) sẽ dịch VNFD đó thành các đối tượng Kubernetes tương ứng. Việc nắm vững chuẩn TOSCA và VNFD/NSD giúp nhóm đảm bảo 3S-COM thiết kế **đúng chuẩn**, dễ dàng tích hợp VNF từ bên thứ ba và tận dụng các **catalog** VNF có sẵn. (Trong thực tế demo, nếu sử dụng OSM, cũng áp dụng khái niệm VNFD/NSD: tạo descriptor cho VNF mẫu trước khi triển khai, chi tiết được trình bày ở phần sau).

4. Các nền tảng MANO nguồn mở

Như đã đề cập, ONAP là lựa chọn chính cho 3S-COM full-featured. Tuy nhiên, ONAP khá đồ sộ; do đó nhóm cũng tham khảo một số giải pháp MANO nguồn mở nhẹ hơn để phục vụ mục đích thử nghiệm và so sánh. Một số nền tảng tiêu biểu: **Open Source MANO (OSM)** – dự án ETSI cung cấp NFVO/VNFM đơn giản; **OpenStack Tacker** – module NFVO+VNFM trong OpenStack; **Open Baton** (Fraunhofer FOKUS) – MANO framework học thuật; **Cloudify** – nền tảng thương mại hỗ trợ MANO đa đám mây. Đặc biệt, **OSM** nổi lên như một lựa chọn phù hợp cho môi trường lab: OSM gộp NFVO và VNFM trong cùng hệ thống, dễ triển khai (có thể chạy dưới dạng container hoặc VM nhẹ) và đã hỗ trợ tích hợp với Kubernetes, OpenStack... OSM sử dụng descriptor định dạng YANG nhưng có thể tương thích logic với TOSCA. đã cài đặt thử OSM phiên bản mới và tìm hiểu cách cấu hình VIM (Kubernetes) cũng như onboard một VNF mẫu lên OSM. Kiến thức này được áp dụng trực tiếp trong Demo 1, khi sử dụng OSM để triển khai VNF thay cho ONAP nhằm đơn giản hóa.

5. Công cụ triển khai tự động (Automation scripts)

Trong trường hợp không sử dụng giải pháp MANO hoàn chỉnh, một cách tiếp cận khác để minh họa nguyên lý NFV Orchestration là sử dụng các công cụ tự động hóa hạ tầng sẵn có như **Ansible, Heat, Terraform**. Các công cụ này cho phép viết kịch bản để tự động tạo máy ảo/container, cấu hình mạng, v.v. Ví dụ, một **Ansible playbook** có thể được viết để: (a) tạo một **Deployment** trên Kubernetes chạy container VNF từ image cho trước, (b) khi cần scale thì tăng replica của Deployment, (c) khi terminate thì xóa Deployment. Kết hợp với một giao diện đơn giản (VD: script CLI hoặc nút bấm trên dashboard), ta có thể mô phỏng chức năng Orchestrator mà không cần hệ thống phức tạp. đã thử nghiệm viết một số kịch bản Ansible cho Kubernetes nhằm hiểu rõ quy trình triển khai tài nguyên bằng code. Tuy giải pháp “tự làm” này không đầy đủ như OSM hay ONAP (thiếu nhiều tính năng quản lý), nhưng nó minh chứng được các bước cơ bản trong quản lý vòng đời VNF, và có thể tích hợp vào Demo 1 nếu OSM gặp hạn chế.

Tóm lại, phần nghiên cứu lý thuyết đã trang bị cho kiến thức tổng quan lẫn chi tiết về NFV-MANO và các công nghệ liên quan. Dựa trên đó, **giải pháp thiết kế cho 3S-COM** được hình thành: tận dụng ONAP + Kubernetes cho kiến trúc đầy đủ, dùng OSM/Kubernetes cho môi trường lab, kết hợp SDN Controller và AI để hoàn thiện tính năng. Phần tiếp theo, báo cáo sẽ trình bày cách thức áp dụng những kiến thức này để thực hiện các nhiệm vụ được giao, bao gồm việc xây dựng tài liệu SRS và triển khai demo minh họa.

VI. Cách giải quyết

Dựa trên yêu cầu bài toán và các giải pháp công nghệ đã phân tích, đã đề xuất phương án triển khai hệ thống 3S-COM theo hai giai đoạn: **(1)** Giai đoạn tài liệu hóa thiết kế và **(2)** Giai đoạn thử nghiệm lab.

1. Giai đoạn 1 – Thiết kế tài liệu (SRS)

Trước tiên, tập trung vào việc **xây dựng SRS cho hệ thống 3S-COM**. Dựa trên kiến thức nền tảng NFV-MANO và định hướng giải pháp (sử dụng ONAP, Kubernetes...), SRS được soạn thảo bao gồm các phần chính: giới thiệu dự án, phạm vi, kiến trúc tổng quan, yêu cầu chức năng/phi chức năng, các trường hợp sử dụng (**use-case**), yêu cầu giao tiếp hệ thống, ràng buộc thiết kế, v.v. Trong đó, đặc biệt làm rõ cách 3S-COM tuân thủ mô hình ETSI NFV:

- **Kiến trúc tổng quan:** SRS mô tả 3S-COM theo kiến trúc tham chiếu NFV với NFVO/VNFM (thông qua ONAP) và VIM (Kubernetes). Sơ đồ kiến trúc logic được đưa ra để minh họa các thành phần chính và luồng tương tác giữa chúng (NFVO <-> VNFM <-> VIM, tích hợp SDN Controller, module AI...). Kiến trúc này đảm bảo mỗi yêu cầu triển khai dịch vụ đều đi qua quy trình: NFVO nhận yêu cầu từ OSS/BSS hoặc người quản trị, kiểm tra tài nguyên qua VIM, yêu cầu VIM tạo VM/container, rồi phối hợp VNFM cấu hình VNF và thiết lập kết nối mạng giữa các VNF. đã sử dụng kiến thức từ chuẩn ETSI và ví dụ ONAP để diễn giải cụ thể một kịch bản triển khai dịch vụ (tương tự như ví dụ NS instantiation trong chuẩn): NFVO đọc **NSD**, cấp phát tài nguyên (qua Kubernetes), gọi VNFM khởi tạo VNF, và kết nối các VNF theo mô hình dịch vụ. *Hình 2* dưới đây (giả định minh họa) thể hiện kiến trúc logic của 3S-COM với các khối chức năng và dòng trao đổi chính.
- **Yêu cầu hệ thống:** Dựa trên phân tích bài toán, liệt kê chi tiết các **yêu cầu chức năng** như đã trình bày (quản lý VNF, dịch vụ mạng, NFVI, SDN, giám sát, AI...) và **yêu cầu phi chức năng** (hiệu năng, bảo mật, mở rộng...). Mỗi yêu cầu trong SRS được diễn đạt rõ, kèm tiêu chí hoàn thành. Ví dụ: *“Hệ thống phải cho phép scale-out VNF theo thời gian thực trong vòng <30 giây kể từ lúc tải tăng vượt ngưỡng”* – đây là yêu cầu hiệu năng cho tính năng mở rộng VNF. Hay *“Hệ thống phải hỗ trợ chuẩn TOSCA VNFD để import VNF từ nhiều nhà cung cấp”* – yêu cầu tương thích chuẩn. Việc định lượng hoặc tham chiếu chuẩn như vậy giúp SRS khả kiểm chứng. đã tham khảo các tài liệu chuẩn (ETSI NFV, ONAP requirements) để đưa ra các con số và tiêu chí phù hợp.
- **Use-case và luồng xử lý:** SRS xây dựng một số kịch bản use-case tiêu biểu, chẳng hạn: Triển khai một dịch vụ mạng mới, Thay đổi quy mô VNF, Xử lý sự cố VNF, Phát hiện tấn công và phản ứng... Mỗi use-case mô tả các bước tương tác giữa người quản trị với hệ thống và phản ứng mong đợi của 3S-COM. Ví dụ, use-case *“Deploy new Network Service”*: Quản trị viên chọn một NS từ catalog -> hệ thống kiểm tra tài nguyên, tạo các VNF (qua VNFM/VIM), cấu hình chúng, báo trạng thái hoàn thành. Các lưu đồ (flowchart) đã được phác thảo cho những luồng này nhằm hỗ trợ thiết kế chi tiết sau này.

- **Ràng buộc và giả định:** SRS cũng nêu rõ những giới hạn và giả định ở giai đoạn đầu. Chẳng hạn, giai đoạn lab sẽ giới hạn số lượng VNF triển khai đồng thời (ví dụ ≤ 10 VNF) và chưa yêu cầu tính HA ngay; nhưng thiết kế phải tính đến khả năng mở rộng về sau. Hoặc giả định rằng các VNF core (3S-Router, 3S-Firewall...) sẽ được phát triển song song và tuân theo format chuẩn, 3S-COM không đi sâu vào nội tại VNF mà coi chúng như hộp đen triển khai. Những điều này đảm bảo phạm vi dự án rõ ràng: 3S-COM tập trung vào *điều phối*, không “sa lầy” vào việc tạo mới chức năng mạng.

Kết quả của giai đoạn 1 là một tài liệu SRS hoàn chỉnh (~30 trang) mô tả yêu cầu và thiết kế cấp cao cho hệ thống 3S-COM. Điều này không chỉ giúp định hướng triển khai cho nhóm phát triển mà còn đóng vai trò là thước đo để so sánh khi xây dựng bản demo.

2. Giai đoạn 2 – Triển khai demo thử nghiệm (Mini-Lab)

Sau khi có thiết kế, tiến hành xây dựng **môi trường thử nghiệm** nhằm chứng minh một số chức năng chính yếu của hệ thống. Như đã vạch ra, thay vì triển khai ngay toàn bộ ONAP (quy mô lớn, phức tạp), nhóm quyết định thực hiện **Demo 1 – Orchestrator NFV thu gọn** với công cụ nhẹ hơn. Phương án cụ thể như sau:

- **Môi trường hạ tầng:** Sử dụng **1 máy chủ vật lý** (PC cá nhân cấu hình CPU 4 nhân, 16GB RAM) cài hệ điều hành Ubuntu 20.04 làm máy chủ lab. Trên đó, triển khai một cụm **Kubernetes** tối giản (dùng **MicroK8s** hoặc **Minikube** cho thuận tiện) để đóng vai trò hạ tầng NFVI. Cụm K8s này về cơ bản có 1 node (all-in-one) chạy container runtime Docker và cung cấp các API để quản lý pods, deployment.
- **Công cụ Orchestrator:** Cài đặt **OSM (Open Source MANO)** phiên bản mới nhất (Release 10) trên máy chủ lab. OSM được triển khai nhanh chóng thông qua container Docker (script cài đặt của OSM tự động thiết lập các container thành phần). Sau khi cài, OSM cung cấp giao diện web UI và các lệnh CLI. cấu hình tích hợp OSM với Kubernetes: thêm Kubernetes cluster của lab làm **VIM account** trong OSM (qua lệnh `osm vim-create`), cho phép OSM có thể điều khiển tạo container trên K8s. (Trong quá trình tích hợp, ta cung cấp cho OSM các thông tin kết nối Kubernetes như URL của API server, token xác thực, etc.).
- **VNF mẫu cho demo:** Chọn triển khai VNF mẫu là **vFirewall** dạng container đơn giản. Ở đây, nhóm sử dụng một container Ubuntu đã cài sẵn iptables để giả lập chức năng firewall (hoặc một container **FRRouting** để làm router ảo). Một descriptor VNFD được tạo cho VNF này theo cú pháp **YANG** của OSM, bao gồm định nghĩa image (sử dụng image Docker trên DockerHub), tài nguyên cần (CPU, RAM), điểm kết nối mạng (VD: một interface kết nối mạng external). Đồng thời, tạo một **NSD** cho dịch vụ mạng chứa một VNF này (đơn giản là dịch vụ chỉ gồm vFirewall). Các descriptor này được nạp vào OSM (qua CLI hoặc GUI).
- **Triển khai VNF qua Orchestrator:** Sử dụng giao diện **OSM Client** (giao diện dòng lệnh hoặc GUI web) để thực hiện lệnh triển khai. Cụ thể, từ dashboard OSM, thao tác: **“Launch NS”** với NSD đã tạo (vFirewall NS). OSM NFVO sẽ tiếp nhận, ánh xạ yêu cầu sang Kubernetes: tạo một **Deployment** trên cluster Kubernetes chạy container vFirewall từ

image chỉ định. Đồng thời, OSM tự động tạo network service (nếu cấu hình, OSM có thể tạo một NetworkAttachmentDefinition cho việc nối mạng pod – tuy demo đơn giản có thể dùng default network). Kết quả, trong vài chục giây, trên Kubernetes xuất hiện một pod mới chạy container vFirewall. OSM hiển thị trạng thái VNF “**running**” cùng địa chỉ IP được cấp.

- **Thao tác quản lý vòng đời:** Để minh họa quản lý vòng đời, thực hiện tiếp các thao tác: **Scale out** VNF – tăng số bản sao vFirewall lên 2 (trong OSM, có thể thực hiện bằng cách chỉnh tham số scaling nhóm VDU nếu descriptor hỗ trợ, hoặc thủ công tạo thêm instance NS). Một cách đơn giản, có thể đăng ký **Scaling Policy** trong VNFD (ví dụ CPU > 70% thì scale) rồi kích hoạt điều kiện giả lập. Kết quả Kubernetes sẽ chạy thêm một pod vFirewall thứ hai, OSM báo 2 VNFs. Sau đó, thực hiện **Scale in** (giảm số instance về 1) – OSM sẽ xóa bớt một pod. Cuối cùng, thử nghiệm **Terminate** – gỡ bỏ dịch vụ: OSM xóa deployment, các container VNF ngừng và giải phóng tài nguyên. Toàn bộ quá trình trên được quan sát qua cả giao diện OSM (trạng thái NS/VNF thay đổi) và Kubernetes dashboard (số pod tăng giảm tương ứng).

Kết quả của Demo 1: từ **giao diện Orchestrator OSM**, quản trị viên có thể nhấn nút “Deploy VNF” và thấy ngay một VNF được tự động khởi tạo trên Kubernetes, sau đó có thể điều khiển **scale out/in** hay terminate chỉ bằng thao tác chuột, thay vì phải tự chạy lệnh kubectl thủ công. Điều này minh chứng rõ nét **nguyên lý hoạt động của 3S-COM NFV** – tức là có một hệ thống điều phối trung gian lo toàn bộ việc tạo/xóa tài nguyên cho VNF trên hạ tầng ảo, người dùng chỉ cần tương tác ở mức dịch vụ. Mặc dù sử dụng OSM thay vì ONAP, demo đã phản ánh **chức năng cốt lõi** mà 3S-COM hướng đến: quản lý lifecycle VNF tự động trên NFVI.

- **Các bước triển khai chi tiết:** Đã viết kịch bản triển khai cho demo để đảm bảo buổi trình bày diễn ra trơn tru. Bao gồm: chuẩn bị sẵn image container vFirewall trên registry (DockerHub), kiểm tra kết nối từ OSM tới Kubernetes (tránh lỗi quyền), tạo trước các descriptor đúng định dạng. Việc cài đặt OSM mất ~15 phút, cấu hình VIM ~5 phút, tạo NS ~5 phút. Toàn bộ demo chạy trên một máy, thời gian deploy VNF ~30 giây, scale ~20 giây. Những con số này cho thấy tính khả thi của giải pháp NFV-MANO mini.
- **Thay thế khác (nếu có):** Trong trường hợp OSM gặp lỗi, nhóm có dự phòng dùng **Ansible script** để triển khai trực tiếp. Kịch bản Ansible tương tự: khi chạy playbook deploy_vFirewall.yaml sẽ gọi module Kubernetes tạo deployment; playbook scale.yaml chỉnh replica; playbook terminate.yaml xóa deployment. Các playbook này có thể gắn vào một giao diện dòng lệnh hoặc web để người dùng bấm nút. Tuy nhiên, may mắn là OSM hoạt động ổn định nên demo đã sử dụng OSM hoàn toàn, giúp thể hiện giao diện trực quan chuyên nghiệp hơn (có bảng trạng thái VNF, log sự kiện của OSM).

Tóm lại, cách giải quyết của là: Em dùng **phương án mô phỏng đơn giản trong lab** (OSM + Kubernetes + VNF mẫu) để kiểm chứng các khái niệm đã thiết kế trong SRS, đồng thời chuẩn bị nền tảng cho việc mở rộng quy mô sau này (chuyển sang ONAP khi đủ nguồn lực). Phần tiếp theo sẽ mô tả cụ thể kết quả đạt được từ quá trình trên và đánh giá những gì còn hạn chế, mở ra hướng phát triển tương lai.

VII. Mô tả phần mềm cài đặt

Phần này trình bày chi tiết về **phần mềm demo** đã được cài đặt và cấu hình trong quá trình thực tập, tương ứng với Demo 1 – Orchestrator NFV mini-lab như đã nêu.

1. Môi trường và công cụ:

- **Hạ tầng phần cứng:** 1 máy PC (Chip Intel Core i5, RAM 16 GB, SSD 256 GB) được sử dụng làm máy chủ cài đặt toàn bộ hệ thống demo.
- **Hệ điều hành & nền tảng ảo hóa:** Cài đặt Ubuntu 20.04 LTS làm OS chính. Trên đó sử dụng **MicroK8s** (bản phân phối Kubernetes tối giản của Canonical) để dựng cụm Kubernetes cục bộ. MicroK8s được chọn vì cài đặt rất nhanh (snap install) và tích hợp sẵn các thành phần cần thiết (Docker daemon, Kubernetes API server, kubectl CLI). Cụm MicroK8s chạy single-node (máy local), với cấu hình mặc định (8 vCPU, 12GB RAM cấp cho MicroK8s từ host).
- **Phần mềm Orchestrator (OSM):** Sử dụng **OSM Release 11** (phiên bản mới, tương thích Kubernetes). OSM được triển khai qua docker-compose: chạy script install_osm.sh (do ETSI OSM cung cấp) để tự động dựng các container OSM (bao gồm các module: NBI – Northbound API, RO – Resource Orchestrator, MON – Monitoring, UI – giao diện, PostgreSQL database...). Quá trình cài OSM mất khoảng 10-15 phút và kết thúc khi giao diện web OSM có thể truy cập tại cổng 80 (<http://<IP>>): giao diện này yêu cầu đăng nhập (mặc định user admin).
- **Cấu hình tích hợp Kubernetes:** Sau khi OSM lên, tiến hành đăng ký Kubernetes Sau đó tạo file **vFirewall_nsd.yaml** mô tả Network Service: - NS gồm 1 VNFD vFirewall ở trên.
- Virtual Link Descriptor: private-net (loại e-line kết nối vFirewall ra ngoài). - Nếu cần, định nghĩa luôn một **scaling group** cho VNFD (scale kích thước 1-3 instance tùy CPU usage).
- **Triển khai dịch vụ vFirewall:** Trên giao diện OSM UI, vào phần **Network Service** -> Launch NS. Chọn NSD = vFirewall-service, chọn VIM = K8sLab, điền tên NS instance. Nhấn **Instantiate**. OSM bắt đầu khởi tạo: qua giao diện có thể thấy log tiến trình (VNFM log). Kubernetes phía dưới cũng bắt đầu tạo pod. Dùng lệnh `microk8s kubectl get pods` thấy xuất hiện pod tên dạng `vfirewall-<id>` trong namespace `osm`. Sau ~20-30s, pod chuyển trạng thái **Running**. Trên OSM UI, NS instance báo **running**, bên trong có 1 VNF vFirewall trạng thái running, kèm địa chỉ IP (do Kubernetes cấp qua Multus hoặc default CNI). VNF đã triển khai thành công.
- **Kiểm tra chức năng VNF:** Để đảm bảo VNF thực sự hoạt động, có thể exec vào container vFirewall (`kubectl exec -it pod/vfirewall-... -- bash`) và chạy thử lệnh `iptables` xem rule có áp dụng. Mặc dù đây ngoài phạm vi Orchestrator, nhưng kiểm tra để chắc chắn VNF mẫu của ta sẵn sàng. Kết quả cho thấy container vFirewall đang chạy Ubuntu với iptables dịch vụ sẵn.
- **Scale out thử nghiệm:** Tiến hành scale VNF. Có hai cách:

- + Nếu NSD đã định nghĩa scaling rule (ví dụ: action manual scale group), có thể dùng lệnh `osm ns-scale --ns <ns_name> --vnf <vnf_name> --scale-out <group_name>`. OSM sẽ tạo thêm 1 pod vFirewall mới.
- + Cách thủ công: tạo một NS instance thứ hai của cùng NSD. (Mỗi NS instance OSM quản lý riêng, nên cách 1 hợp lý hơn).
- + Ở đây, thử cách 1: chạy lệnh scale-out, OSM log hiển thị đang tạo thêm VDU. Kubernetes xuất hiện pod thứ 2 (tên khác, do OSM create). Sau ~20s, có 2 pod vFirewall running. Trên OSM UI, trong NS instance now hiển thị 2 VNFs (cùng type vFirewall). Như vậy scale-out thành công. Sau đó thử scale-in: `osm ns-scale --ns <name> --scale-in <group_name>` – OSM chọn remove một instance, Kubernetes pod thứ 2 bị xóa, trả về 1 pod. Trạng thái NS trở lại ban đầu.
- **Kết thúc dịch vụ:** Thực hiện lệnh `osm ns-delete <ns_name>` hoặc dùng UI “Terminate”. OSM sẽ gỡ toàn bộ NS: pod vFirewall còn lại biến mất, OSM xóa record NS instance. Tài nguyên được giải phóng sạch sẽ.

2. Đánh giá hoạt động

Demo chạy thành công, minh họa được các chức năng chính: **triển khai tự động VNF** và **thay đổi quy mô VNF** trên Kubernetes thông qua Orchestrator OSM. Thời gian phản hồi cho các thao tác khá nhanh (vì môi trường nhỏ gọn). Giao diện OSM cung cấp thông tin trực quan: trạng thái VNF, log scaling... giúp người xem dễ hình dung quy trình. Mặc dù OSM không hoàn toàn giống ONAP (về kiến trúc và tính năng phụ), nhưng trong phạm vi chức năng cơ bản, OSM thực hiện đúng vai trò NFVO/VNFM quản lý VNF container.

Một số hạn chế nhỏ được ghi nhận: - OSM hiện tại hỗ trợ Kubernetes nhưng cần cấu hình plugin mạng (Multus) nếu muốn phức tạp hơn (nhiều interface cho pod). Demo đơn giản dùng 1 interface nên chưa gặp vướng mắc. - Việc cấu hình IP cho VNF container do Kubernetes cấp tự động (DHCP), OSM chưa can thiệp được sâu như ONAP. Nếu yêu cầu IP cố định có thể phải tạo thêm config script. - OSM không tích hợp sẵn SDN hay AI gì, nên demo chưa thể hiện phần SDN/AI của 3S-COM. Những phần này sẽ cần demo riêng hoặc chờ khi ONAP được dựng lên (ONAP có DCAE và Policy hỗ trợ AI, cũng như module SDN-C cho SDN Controller).

Nhìn chung, phần mềm cài đặt demo đã đáp ứng mục tiêu: xây dựng một **môi trường NFV thu nhỏ** với đầy đủ các thành tố (NFVI + Orchestrator + VNF) để kiểm chứng ý tưởng thiết kế 3S-COM.

VIII. Kết quả đạt được và hướng phát triển

Qua quá trình thực tập và thực hiện các nhiệm vụ trên, đã thu được những kết quả nhất định, đồng thời rút ra các bài học kinh nghiệm quý báu. Dưới đây là tóm tắt **kết quả đạt được** và một số định hướng phát triển sau dự án:

1. Kết quả đạt được:

- **Hoàn thiện cơ sở lý thuyết và tài liệu nền tảng:** đã nghiên cứu sâu về các công nghệ trụ cột (NFV, SDN, TOSCA, AI/IDS...) và biên soạn thành công tài liệu “Kiến thức nền tảng cho hệ thống 3S-COM”. Tài liệu này tổng hợp những kiến thức cập nhật, bám sát vào nhu cầu của dự án, làm nền tảng giúp nhóm phát triển hiểu rõ bối cảnh kỹ thuật. Đặc biệt, nhờ nắm vững kiến trúc NFV-MANO và các chuẩn ETSI liên quan, có thể áp dụng trực tiếp vào việc thiết kế hệ thống.
- **Xây dựng đặc tả yêu cầu (SRS) cho 3S-COM:** Một kết quả quan trọng là tài liệu SRS (~40 trang) đã được hoàn thiện, mô tả đầy đủ kiến trúc và yêu cầu hệ thống. SRS này đóng vai trò định hướng cho toàn dự án CoreRouter, đảm bảo các bước phát triển sau tuân theo đúng mục tiêu đã đề ra. SRS cũng đã được nhóm hướng dẫn và các bên liên quan xem xét, phản hồi tích cực về tính logic và khả thi. Đây là cơ sở để ban lãnh đạo dự án tự tin triển khai các giai đoạn kế tiếp.
- **Triển khai thành công demo minh họa Orchestrator NFV:** đã thiết lập được một môi trường lab hoạt động và chạy thử **demo Orchestrator NFV (Demo 1)** như kế hoạch. Kết quả demo cho thấy hệ thống có thể tự động triển khai VNF và quản lý vòng đời (scale, terminate) thông qua một giao diện điều phối trung tâm. Điều này minh chứng tính đúng đắn của kiến trúc 3S-COM ở mức độ cơ bản, giúp giảng viên hướng dẫn và các thành viên dự án có cái nhìn trực quan hơn về sản phẩm tương lai. Sau demo, các thầy/cô đã đánh giá cao nỗ lực và kết quả thực nghiệm, đồng thời góp ý một số cải tiến (như tích hợp thêm hiển thị tài nguyên real-time trên giao diện).
- **Thu thập số liệu và kinh nghiệm thực tế:** Thông qua demo, nhóm cũng thu thập được một số số liệu thực nghiệm như thời gian khởi tạo VNF, mức sử dụng tài nguyên của Orchestrator (OSM tiêu tốn ~2GB RAM, ~5% CPU khi idle), khả năng mở rộng tối đa trên máy lab (có thể chạy ~15 container VNF đồng thời trước khi CPU quá tải). Những thông tin này hữu ích để hiệu chỉnh thiết kế cho phù hợp thực tế (ví dụ cân nhắc cấu hình phần cứng khi triển khai ONAP thật, ước lượng tài nguyên cần cho môi trường production).
- **Khẳng định hướng tiếp cận:** Qua việc khảo sát nền tảng tương tự (ONAP, OSM, ODL...) và thử nghiệm demo, dự án càng củng cố niềm tin rằng hướng đi đã chọn (kết hợp NFV + SDN + AI) là phù hợp xu hướng. Các giải pháp tương tự trên thế giới như ONAP, ETSI OSM cũng cho thấy mô hình kiến trúc mở và dùng cloud-native là đúng đắn. Kết quả nghiên cứu so sánh này được trình bày trong nhóm dự án và được thống nhất ủng hộ. Đây là thành công gián tiếp nhưng rất quan trọng, vì nó đảm bảo dự án không đi chệch hướng so với dòng chảy công nghệ.

2. Hạn chế và hướng phát triển

Bên cạnh các kết quả trên, cũng nhận thấy một số hạn chế trong phạm vi thực tập và đề xuất hướng phát triển để hoàn thiện hệ thống:

- **Hạn chế về phạm vi demo:** Demo 1 mới chỉ tập trung vào chức năng NFV Orchestration (triển khai VNF cơ bản). Các khía cạnh khác của 3S-COM như tích hợp SDN, phân tích AI, hay giao diện người dùng tùy biến vẫn chưa được hiện thực trong demo do giới hạn thời gian. Do đó, một **hướng phát triển trước mắt** là xây dựng thêm các demo bổ trợ: ví dụ **Demo 2** về giao diện dashboard (mô phỏng portal quản trị 3S-COM) và **Demo 3** về tích hợp SDN cơ bản (dùng mininet + ONOS để cho thấy điều khiển luồng mạng). Những demo này sẽ giúp hoàn thiện bức tranh toàn diện về hệ thống trước khi bước vào triển khai phiên bản đầy đủ.
- **Chuyển từ OSM sang ONAP (bản Lite):** Mặc dù OSM đáp ứng tốt cho lab, nhưng để tiến tới sản phẩm thật, 3S-COM cần chuyển sang dùng **ONAP** (như thiết kế đặt ra) để tận dụng đầy đủ tính năng và đảm bảo khả năng mở rộng. ONAP có thể năng, vì vậy nhóm dự án dự kiến triển khai một phiên bản “**3S-COM Lite**” dùng ONAP với cấu hình tinh giản (có thể tắt bớt một số microservice chưa cần thiết, giới hạn số VNF) để thử nghiệm trong lab mở rộng. Đây là bước chuyển quan trọng, đòi hỏi tìm hiểu sâu về cài đặt ONAP (sử dụng **OOM – ONAP Operations Manager** trên Kubernetes) và cấu hình ONAP tích hợp với Kubernetes cluster. sau thực tập có thể tiếp tục tham gia hỗ trợ quá trình này, áp dụng những kiến thức ONAP đã nghiên cứu được.
- **Phát triển các VNF nội bộ (3S-series):** SRS đề cập việc các VNF như **3S-Router, 3S-Firewall, 3S-IDPS** sẽ được phát triển riêng và tích hợp vào 3S-COM. Hiện tại, demo dùng VNF mẫu từ nguồn có sẵn (Ubuntu iptables). Hướng tới, nhóm sẽ bắt tay xây dựng các VNF “made in Vietnam” này, tối ưu cho nhu cầu mạng lõi. Ví dụ, 3S-Router có thể dựa trên FRRouting nhưng bổ sung tính năng đặc thù; 3S-Firewall có thể tùy biến trên nền tảng nftables hoặc IDS Suricata kết hợp AI. Khi các VNF này sẵn sàng, 3S-COM sẽ cần cập nhật để hỗ trợ chúng (ví dụ bổ sung param cấu hình đặc thù khi instantiate). kiến nghị lập một **nhóm phụ trách VNF** tách biệt với nhóm Orchestrator, nhưng phối hợp chặt để đảm bảo tương thích (thống nhất format VNFD, API quản lý).
- **Tích hợp module AI/ML cho IDS:** Một điểm độc đáo của 3S-COM là tích hợp AI cho an ninh. Hiện chức năng này mới dừng ở mức yêu cầu thiết kế. Hướng phát triển là nghiên cứu các mô hình học máy phù hợp (như dùng mạng Neural phát hiện bất thường dựa trên dataset CIC-IDS2017) và triển khai thử trong môi trường lab. ONAP có khung DCAE (Data Collection, Analytics, Events) có thể tận dụng để xử lý dữ liệu AI. Trong giai đoạn tiếp theo, dự án sẽ cần nhân lực chuyên về dữ liệu để xây dựng mô hình AI và huấn luyện, sau đó tích hợp ngõ ra (output) của mô hình với Policy Framework của ONAP để tự động thực thi phản ứng. thực tập đã đề xuất một kịch bản mẫu: thu thập log tấn công từ vIDS, dùng mô hình phát hiện DDoS, nếu phát hiện thì ONAP Policy sẽ trigger scale-out vFirewall. Đây sẽ là mục tiêu thách thức nhưng rất giá trị cho 3S-COM.
- **Hoàn thiện giao diện người dùng (Portal):** Cuối cùng, hệ thống cần một giao diện portal thân thiện để tổng hợp mọi chức năng (thay vì dùng giao diện mặc định của ONAP hay OSM vốn khá phức tạp). Hướng phát triển là xây dựng **3S-COM Portal** – một ứng dụng web cho phép quản trị mạng thực hiện các thao tác thiết kế dịch vụ, theo dõi trạng thái mạng, nhận cảnh báo bảo mật... tất cả trong một bảng điều khiển. Nhóm dự án có kế hoạch

dùng các công nghệ web hiện đại (React, Angular cho front-end; backend có thể sử dụng các API REST của ONAP) để phát triển portal này. Trong giai đoạn demo, một prototype nhỏ (Demo 2 đề cập ở trên) đã được đề xuất như bước đầu. trong đợt thực tập đã đóng góp ý tưởng và bản phác thảo giao diện, và có thể tiếp tục tham gia phát triển phần này nếu có điều kiện.

Tóm lại, kết quả thực tập đã đáp ứng các mục tiêu ban đầu đề ra: từ nghiên cứu, tài liệu đến triển khai demo. Tuy còn nhiều hạng mục cần tiếp tục, nhưng nhờ những nền tảng đã xây dựng, dự án 3S-COM có một lộ trình rõ ràng để tiến tới hoàn thiện. Phần tiếp theo sẽ tổng kết lại những kiến thức và kỹ năng đã tích lũy được qua đợt thực tập này.

IX. Kỹ năng & kiến thức thu thập được

Quá trình tham gia dự án CoreRouter và thực hiện các nhiệm vụ nói trên đã giúp gặt hái được nhiều kiến thức chuyên môn và kỹ năng nghề nghiệp quý báu. Cụ thể:

- **Kiến thức chuyên sâu về NFV/SDN:** nắm vững kiến trúc NFV-MANO, hiểu rõ vai trò từng thành phần (NFVO, VNFM, VIM) và quy trình quản lý dịch vụ mạng ảo hóa. Đồng thời, cũng hiểu nguyên lý SDN (tách control/data plane) và cách tích hợp SDN Controller vào hệ thống mạng. Những kiến thức này khá mới mẻ và chuyên sâu, giúp có nền tảng vững chắc trong lĩnh vực mạng viễn thông hiện đại.
- **Hiểu biết về các nền tảng orchestration thực tế:** Thông qua việc nghiên cứu và cài đặt thử, có kinh nghiệm thực tiễn với các công cụ như **ONAP, OSM, Kubernetes**. Đặc biệt, việc trực tiếp cấu hình OSM và Kubernetes trong demo đã rèn luyện kỹ năng triển khai hệ thống phân tán, quản lý container, cũng như khắc phục các sự cố cài đặt (networking, credentials) – những trải nghiệm quý giá khó có được nếu chỉ học lý thuyết.
- **Kỹ năng thiết kế hệ thống & viết tài liệu kỹ thuật:** Việc soạn thảo tài liệu SRS và nền tảng lý thuyết đã nâng cao đáng kể kỹ năng trình bày vấn đề một cách mạch lạc, logic. học được cách phân tích yêu cầu, đặc tả hệ thống theo chuẩn IEEE SRS, biết cách lập luận dựa trên số liệu và dẫn chứng khi viết tài liệu. Ngoài ra, kỹ năng lập dàn ý, định dạng văn bản và vẽ biểu đồ kiến trúc cũng được cải thiện thông qua quá trình làm báo cáo.
- **Kỹ năng nghiên cứu độc lập:** Dự án đòi hỏi tự tìm hiểu nhiều công nghệ mới. Nhờ vậy, rèn luyện được khả năng **tự nghiên cứu tài liệu tiếng Anh**, từ việc đọc tiêu chuẩn ETSI, tài liệu ONAP, đến tra cứu các blog kỹ thuật, tutorial. Kỹ năng tra cứu nhanh (Google search hiệu quả, đọc chọn lọc) và tổng hợp thông tin hữu ích từ nguồn lớn đã được trau dồi qua từng nhiệm vụ.
- **Kỹ năng quản lý thời gian và lập kế hoạch:** Với khối lượng công việc khá lớn (nghiên cứu, viết tài liệu, làm demo) trong thời gian thực tập giới hạn, học cách **lên kế hoạch** và phân chia thời gian hợp lý cho từng nhiệm vụ. Việc đặt các mốc hoàn thành (ví dụ: 2 tuần xong draft SRS, tuần tiếp theo làm demo) giúp công việc trôi chảy và không bị dồn vào

cuối kỳ. Đây là kỹ năng mềm quan trọng trong môi trường làm việc thực tế, nơi phải cân bằng nhiều công việc song song.

- **Làm việc nhóm và giao tiếp:** Mặc dù mỗi thành viên phụ trách mảng riêng, vẫn thường xuyên **tương tác nhóm** để trao đổi kết quả nghiên cứu, thống nhất các quyết định thiết kế. Qua đó, kỹ năng trình bày ý tưởng, thảo luận phản biện và phối hợp với người khác được nâng cao. cũng học được cách tiếp thu ý kiến đóng góp từ mentor và đồng đội, chỉnh sửa sản phẩm (tài liệu, demo) cho hoàn thiện hơn.
- **Kỹ năng kỹ thuật khác:** Bên cạnh các kỹ năng chính, còn thuần thục hơn trong một số công cụ/hệ thống: sử dụng Linux ở mức quản trị (cài dịch vụ, chỉnh cấu hình, xử lý lỗi cài đặt), sử dụng Git để quản lý version tài liệu và code demo, sử dụng Markdown và các công cụ soạn thảo (Word, draw.io cho vẽ sơ đồ)...

Nhìn chung, đợt thực tập không chỉ giúp em áp dụng kiến thức đã học mà còn mở rộng hiểu biết sang các mảng công nghệ mới, đồng thời rèn luyện nhiều kỹ năng mềm cần thiết cho sự nghiệp sau này. Đây là bước đệm quan trọng để tự tin hơn khi tham gia các dự án thực tế trong lĩnh vực viễn thông – CNTT.

X. Hướng phát triển tiếp theo để hoàn thiện hệ thống

Dự án 3S-COM nói riêng và CoreRouter nói chung là một hành trình dài hạn, đòi hỏi tiếp tục phát triển qua nhiều giai đoạn để đạt được phiên bản hoàn thiện. Từ những nền tảng đã xây dựng trong giai đoạn nghiên cứu và demo, các bước tiếp theo đề xuất để tiến tới **hệ thống hoàn chỉnh (bản “Full”)** bao gồm:

1. **Triển khai phiên bản 3S-COM Lite trên môi trường lab mở rộng:** Như đề cập, bước kế tiếp sẽ là cài đặt **ONAP** trên cụm Kubernetes lab và cấu hình tích hợp đầy đủ các module cần thiết (NFVO, VNFM, SDN-C, DCAE...). Song song, triển khai các VNF mẫu trên Kubernetes để ONAP quản lý (có thể dùng các mẫu VNFs của ONAP như vFW, vDNS ban đầu). Mục tiêu giai đoạn này là kiểm tra khả năng tương thích của ONAP với hạ tầng Kubernetes local và điều chỉnh tham số cho phù hợp (vì ONAP thường mặc định cho OpenStack VIM). Kết quả mong đợi là một hệ thống 3S-COM Lite chạy được end-to-end: từ thiết kế dịch vụ trên ONAP SDC, triển khai VNF qua SO + Kubernetes, giám sát qua DCAE, điều khiển SDN qua SDN-C... Dựa vào đó, hiệu chỉnh lại kiến trúc nếu phát hiện nhược điểm (ví dụ: nếu ONAP quá nặng, có thể tính phương án thay thế một số module bằng bản nhẹ hơn).
2. **Phát triển tính năng bảo mật chủ động:** Tích hợp mô-đun **AI/ML** vào luồng hoạt động của 3S-COM. Cụ thể, tiến hành xây dựng thử nghiệm hệ thống phát hiện xâm nhập: thu thập dữ liệu (logs từ VNF, số liệu hạ tầng) đưa vào một pipeline phân tích (có thể dùng DCAE Analytics Apps của ONAP hoặc tự phát triển service tách ngoài), ứng dụng mô hình học máy đã huấn luyện để nhận diện bất thường. Sau đó, thiết lập **chính sách (Policy)** trong ONAP để khi nhận cảnh báo từ AI, tự động thực hiện hành động phòng thủ (như scale-out, chặn luồng qua SDN). Giai đoạn đầu có thể sử dụng các dataset có sẵn (CIC-IDS2017,

UNSW-NB15) để huấn luyện offline, kiểm thử với tình huống giả lập tấn công nhỏ. Về lâu dài, cần tối ưu mô hình AI để chạy real-time với luồng dữ liệu lớn trong mạng thực.

3. **Tích hợp thiết bị và công nghệ mới:** Khi mở rộng ra triển khai thực tế, 3S-COM sẽ phải làm việc với các phần tử mạng cụ thể. Ví dụ: tích hợp điều khiển các **router P4** thật qua giao diện SDN Controller – do đó cần phát triển adapter để 3S-COM (qua ONAP SDN-C hoặc một microservice riêng) nói chuyện được với controller (như ONOS/ODL) và push cấu hình xuống thiết bị. Tương tự, nếu sử dụng hạ tầng multi-cloud (vừa có Kubernetes, vừa có OpenStack hoặc cloud public), hệ thống phải tích hợp nhiều VIM đồng thời và xử lý bài toán đa miền (multi-domain orchestration). Đây là hướng phức tạp, có thể tận dụng khả năng mở rộng của ONAP (Multi-cloud plugin) nhưng chắc chắn cần thử nghiệm nhiều để đảm bảo mọi thứ thông suốt. Ngoài ra, nên cân nhắc công nghệ **Service Mesh** (như Istio) cho việc quản lý giao tiếp giữa các microservices 3S-COM nếu hệ thống ngày càng lớn.
4. **Phát triển các VNF/tính năng mới:** Hoàn thiện các **3S-VNF nội bộ** (3S-Router, 3S-Firewall, 3S-IDPS) với đầy đủ tính năng theo yêu cầu mạng lõi. Triển khai chúng trên môi trường container/VM tùy trường hợp (ví dụ 3S-Router hiệu năng cao có thể cần chạy trên VM với DPDK để tăng tốc). Cập nhật các descriptor và quy trình quản lý cho các VNF này trong 3S-COM. Song song, phát triển thêm các tính năng mở rộng: hỗ trợ **VNFFG (VNF Forwarding Graph)** – chuỗi dịch vụ; tích hợp **network slicing** (nếu hướng tới 5G core), hỗ trợ **edge computing** (triển khai VNF phân tán ở nhiều site)... Mỗi tính năng sẽ thêm độ phức tạp cho Orchestrator, nhưng kiến trúc mở ETSI NFV sẽ giúp định hướng cách thêm vào (ví dụ VNFFG tương ứng với việc 3S-COM quản lý thêm chiều kết nối giữa các VNF theo đồ thị, ONAP đã có module ARIA/SDN-C có thể hỗ trợ).
5. **Hoàn thiện giao diện và trải nghiệm người dùng:** Xây dựng **portal quản trị 3S-COM** hoàn chỉnh, tích hợp tất cả chức năng vào giao diện web duy nhất. Giao diện này cần thân thiện với quản trị mạng – có thể thiết kế dạng **dashboard** trực quan với biểu đồ, bản đồ mạng, và các menu thao tác rõ ràng (triển khai dịch vụ mới, xem nhật ký, quản lý sự cố...). Thực hiện các thử nghiệm trải nghiệm (UX test) với người dùng giả lập để tinh chỉnh giao diện. Đồng thời, bổ sung hệ thống **phân quyền người dùng** trên portal (quản trị viên vs người dùng thường, phân quyền theo tenant...). Mục tiêu là khi bàn giao 3S-COM, các kỹ sư mạng có thể sử dụng hệ thống mà không cần hiểu biết sâu về ONAP hay NFV bên dưới.
6. **Kiểm thử và tối ưu hiệu năng, độ ổn định:** Trước khi triển khai thực tế, tiến hành một loạt **bài kiểm thử** trong môi trường lab mô phỏng tải nặng: ví dụ triển khai 100 VNF cùng lúc, giả lập node vật lý bị lỗi, tấn công DDoS vào một VNF... để đánh giá khả năng chịu tải và ổn định của 3S-COM. Từ đó tối ưu các tham số (số thread trong NFVO, cơ chế retry khi gọi API VIM, dung lượng queue xử lý sự kiện...). Nếu phát hiện nút thắt cổ chai, có thể nâng cấp phần cứng hoặc scale-out các thành phần của chính 3S-COM (ví dụ chạy ONAP phân tán nhiều node). Mục tiêu là đảm bảo hệ thống đáp ứng các tiêu chí hiệu năng đã đề ra trong SRS khi hoạt động thật.
7. **Triển khai pilot trên mạng thực tế:** Cuối cùng, sau khi các bước trên hoàn thành, thực hiện triển khai pilot 3S-COM trên một môi trường gần với thực tế (có thể tại lab của đối

tác viễn thông, kết nối với thiết bị thật). Chạy thử một số use-case quản lý mạng lõi (triển khai một dịch vụ VPN ảo cho khách hàng chẳng hạn) và theo dõi sát sao. Thu thập phản hồi từ kỹ sư vận hành thực tế để điều chỉnh lần cuối. Sau pilot, dự án có thể tự tin triển khai diện rộng cho hệ thống core network sản phẩm.

Những hướng phát triển trên trải rộng nhiều mảng từ kỹ thuật đến trải nghiệm người dùng. Điều này cho thấy việc xây dựng một hệ thống như 3S-COM là một quá trình liên tục và liên ngành. Kết thúc đợt thực tập, đã góp phần tạo nền móng vững chắc cho dự án; chặng đường tiếp theo tuy nhiều thách thức nhưng hứa hẹn sẽ đem lại một giải pháp **core network orchestration** hiện đại, **linh hoạt** và **an toàn**, đáp ứng tầm nhìn của dự án CoreRouter.

XI. Tài liệu tham khảo

1. **ETSI** – *Network Functions Virtualisation (NFV); Management and Orchestration*, ETSI GS NFV-MAN 001 V1.1.1, 2014. (Kiến trúc tổng quan NFV-MANO của ETSI, định nghĩa NFVO, VNFM, VIM và các giao diện)
2. **ETSI** – *NFV Release 2; NFV-SOL001 – NFV descriptors based on TOSCA*, ETSI GS NFV-SOL001 V2.6.1, 2019. (Quy định chi tiết về cấu trúc thông tin VNFD/NSD dùng TOSCA YAML trong môi trường NFV)
3. **Linux Foundation** – *ONAP Documentation – Architecture Overview*, 2023. (Tài liệu kiến trúc ONAP, mô tả các thành phần SO, SDN-C, VNFM, Data Analytics trong ONAP và tích hợp Kubernetes)
4. **ETSI OSM** – *OSM Release 17 Documentation*, ETSI, 2023. (Hướng dẫn cài đặt và sử dụng Open Source MANO, bao gồm cách tích hợp OSM với Kubernetes để triển khai CNF)
5. **Jennifer English** – *What is NFV MANO? Network functions virtualization management and orchestration*, TechTarget, Apr 2025. (Bài viết giới thiệu dễ hiểu về NFV-MANO và các thành phần chính trong kiến trúc NFV)
6. **Studocu** – *NFV Part 2 – NFV Architecture Overview and Key Components*, 2022. (Tài liệu tham khảo tiếng Việt về kiến trúc NFV, giải thích chức năng NFVO, VNFM, VIM và liệt kê một số giải pháp MANO nguồn mở như Tacker, OSM)
7. **Báo cáo nội bộ dự án CoreRouter** – *Nền tảng lý thuyết cho hệ thống 3S-COM*, 2025. (Tài liệu do nhóm thực tập biên soạn, trình bày chi tiết kiến trúc NFV-MANO, tích hợp SDN, mô hình TOSCA và ứng dụng AI cho 3S-COM)
8. **Báo cáo nội bộ dự án CoreRouter** – *Đặc tả Yêu cầu Hệ thống (SRS) 3S-COM*, 2025. (Tài liệu SRS của hệ thống 3S-COM, chứa kiến trúc tổng quan ONAP+K8s, yêu cầu chức năng/phi chức năng và các use-case mẫu cho hệ thống)
9. **Báo cáo thực tập CoreRouter** – *Thiết kế hệ thống 3S-COM và Demo Orchestrator NFV*, 2025. (Bản báo cáo nhóm tóm tắt kết quả nghiên cứu, thiết kế SRS, lộ trình phát triển và đề xuất demo minh họa cho hệ thống 3S-COM)
10. **Light Reading** – *CableLabs Hails Major ETSI NFV Milestone*, M. Silbey, 2017. (Bài viết giới thiệu bộ chuẩn ETSI NFV-SOL001–SOL005 và tầm quan trọng của chúng trong việc tạo hệ sinh thái NFV mở)

Ý kiến đánh giá:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm số: Điểm chữ:

Hà Nội, ngày tháng năm 20 .

Giảng viên đánh giá

(Ký, ghi rõ họ tên)